

Partie 1 – Initialisation et parsing simple

```
from tree_sitter import Language, Parser
import tree_sitter_java as tsjava

# Utilisation de la nouvelle syntaxe

# Initialisation correcte de Tree-sitter pour Java
JAVA_LANGUAGE = Language(tsjava.language())
parser = Parser(JAVA_LANGUAGE)

code = b'''
public class Test {
    public static void main(String[] args) {
        int x = 3 + 4 * 5;
        System.out.println(x);
    }
}
'''

tree = parser.parse(code)
root_node = tree.root_node
print(root_node.sexp())
```

Partie 2 – Exploration de l'arbre de syntaxe

```
def explore(node, code_bytes, indent=0):
    prefix = " " * indent
    snippet = code_bytes[node.start_byte:node.end_byte].decode("utf-8").strip()
    print(f"{prefix}- {node.type}: {snippet}")
    for child in node.children:
        explore(child, code_bytes, indent + 1)

explore(root_node, code)
```

Extrait d'affichage attendu :

```
- program: public class Test { ... }
- class_declaration: class Test { ... }
- method_declaration: public static void main(String[] args) { ... }
- variable_declaration: int x = 3 + 4 * 5;
- binary_expression: 3 + 4 * 5
- integer_literal: 3
- +
- binary_expression: 4 * 5
- integer_literal: 4
- *
- integer_literal: 5
```

Partie 3 – Extraction d'opérations binaires

```
def extract_binary_operations(node, code_bytes, ops=[]):
    if node.type == "binary_expression":
```

```

left = node.child_by_field_name('left')
operator = node.children[1]
right = node.child_by_field_name('right')

def get_expr(n):
    if n.type == 'binary_expression':
        return extract_binary_operations(n, code_bytes, [])
    return code_bytes[n.start_byte:n.end_byte].decode("utf-8")

op_dict = {
    'op':
code_bytes[operator.start_byte:operator.end_byte].decode("utf-8"),
    'left': get_expr(left),
    'right': get_expr(right)
}
ops.append(op_dict)

for child in node.children:
    extract_binary_operations(child, code_bytes, ops)
return ops

# Utilisation :
binary_ops = extract_binary_operations(root_node, code)
print(binary_ops)

```

Sortie attendue :

```

[
  {
    'op': '+',
    'left': '3',
    'right': [
      {
        'op': '*',
        'left': '4',
        'right': '5'
      }
    ]
  }
]

```

Partie 4 – Questions de compréhension (propositions de réponses)

1. **Différence AST Tree-sitter vs AST compilateur** : Tree-sitter génère un arbre de syntaxe proche du code source, utilisé pour des outils interactifs. Un compilateur construit souvent un AST plus abstrait, optimisé pour la génération de code.
2. **Utilité dans les outils modernes** : Fournit un AST incrémental, rapide, utilisable dans les IDE pour le surlignage syntaxique, la complétion, la navigation...
3. **Erreurs détectables** : Utilisation de variables non déclarées, expressions invalides, structures incorrectes, opérations interdites, etc.